# Pandas Quick Reference

Class notes cheat sheet: DataFrames, selecting, cleaning, statistics, apply(), dates and groupby

Built from pandas.ipynb and pandas_2.ipynb. All code was run and tested with pandas 3.0.

## How to use this sheet

- Every example assumes `import pandas as pd`. Sections that also need NumPy show `import numpy as np`.
- In Jupyter, the last line of a cell is displayed automatically. In a normal .py file, wrap it in `print()`. Single numbers are shown with `print()` here, so you see `84.0` and not `np.float64(84.0)`.
- Outputs were produced by running the code. Long tables are shortened with `...`. Random results (`sample()`) will be different on your computer.
- **Syntax** boxes show the pattern to remember. Grey-blue boxes are useful notes. Amber boxes mark **corrections** to your class notes and **common mistakes**.
- Variable names (`df`, `df1`, `copy`, `instructor_group`, ...) are the same as in your class notes.
- If your pandas version is older, a few displays differ (for example `object` instead of `str`). The code works the same.

## Data files used in the examples

| File | What it contains | Sections |
|---|---|---|
| `student_data.csv` | 20 students: StudentID, FullName, Data Structure Marks, Algorithm Marks, Python Marks, CompletionStatus, EnrollmentDate, Instructor, Location. From Section 11 on, this file also has one extra, completely empty row at the end (as in your second notebook). | 1 to 16 |
| `phitron_student_marks.xlsx`, `students.parquet` | The same student table saved as an Excel file and a Parquet file. | 2 |
| `data.json` | 3 students with StudentID, FullName and the three marks columns. | 2 |
| `student_completed_data.csv` | 20 students who finished the course: enrollment and finish dates, instructor, location and Total Marks. | 17, 18 |

## Contents


1. DataFrame and Series
2. Loading Data from Files
3. Inspecting a DataFrame
4. Creating a DataFrame from Python Data
5. Selecting Data
6. Index, iloc and Renaming Columns
7. Changing Data
8. Looping Through Rows
9. Sorting
10. Filtering Rows
11. Adding New Columns
12. Unique Values: unique() and nunique()
13. Missing (Null) Values
14. Duplicate Values
15. Statistical Functions
16. apply()
17. Dates and Time Differences
18. groupby()

# 1. DataFrame and Series

A **DataFrame** is a table with rows and columns (a 2D grid). A **Series** is one single column (or one single row) of a DataFrame.

## 1.1 DataFrame

```
import pandas as pd

# DataFrame -> tabular data (2D grid)
df = pd.read_csv('student_data.csv')
print(df)
```

OUTPUT

```
   StudentID          FullName  ...  Instructor    Location
0     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
1     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
2     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
..       ...               ...  ...         ...         ...
16    PH1017       Afsana Mimi  ...   Mr. Karim       Dhaka
17    PH1018       Babul Ahmed  ...   Ms. Salma      Sylhet
18    PH1019      Faria Rahman  ...   Mr. David  Chattogram
19    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

[20 rows x 9 columns]
```

```
print(type(df))
```

OUTPUT

```
<class 'pandas.DataFrame'>
```

## 1.2 Series

```
# Series -> a single column (or a single row)
student_id = df['StudentID']
print(student_id)
```

OUTPUT

```
0     PH1001
1     PH1002
2     PH1003
3     PH1004
       ...
16    PH1017
17    PH1018
18    PH1019
19    PH1020
Name: StudentID, Length: 20, dtype: str
```

```
print(type(student_id))
```

OUTPUT

```
<class 'pandas.Series'>
```

**Note:** One pair of square brackets with one column name, `df['StudentID']`, gives a **Series**. The Series keeps the same index (0, 1, 2, ...) as the DataFrame.

**Note:** Pandas 3 prints the class names as `pandas.DataFrame` and `pandas.Series`. Older versions print the longer names `pandas.core.frame.DataFrame` and `pandas.core.series.Series`. They mean the same thing.

# 2. Loading Data from Files

Pandas has one reader function for each file type. Each one returns a DataFrame.

| File type | Function | Needs extra package |
|---|---|---|
| CSV | `pd.read_csv('file.csv')` | none |
| Excel | `pd.read_excel('file.xlsx')` | openpyxl |
| Parquet | `pd.read_parquet('file.parquet')` | pyarrow |
| JSON | `pd.read_json('file.json')` | none |

## 2.1 CSV file

```
csv_data = pd.read_csv('student_data.csv')
print(csv_data)
print(type(csv_data))
```

OUTPUT

```
   StudentID          FullName  ...  Instructor    Location
0     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
1     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
2     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
..       ...               ...  ...         ...         ...
16    PH1017       Afsana Mimi  ...   Mr. Karim       Dhaka
17    PH1018       Babul Ahmed  ...   Ms. Salma      Sylhet
18    PH1019      Faria Rahman  ...   Mr. David  Chattogram
19    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

[20 rows x 9 columns]
<class 'pandas.DataFrame'>
```

## 2.2 Excel file

```
excel_file = pd.read_excel('phitron_student_marks.xlsx')
print(excel_file)
print(type(excel_file))
```

*Output: the same student table as the CSV, followed by the DataFrame class line.*

## 2.3 Parquet file

```
parquet_file = pd.read_parquet('students.parquet')
print(parquet_file)
print(type(parquet_file))
```

*Output: the same student table as the CSV, followed by the DataFrame class line.*

## 2.4 JSON file

```
json_file = pd.read_json('data.json')
print(json_file)
print(type(json_file))
```

OUTPUT

```
  StudentID          FullName  Data Structure Marks  Algorithm Marks  Python Marks
0    PH1001       Alif Rahman                    85               85            88
1    PH1002     Fatima Akhter                    92               92            90
2    PH1004  Jannatul Ferdous                    78               78            82
<class 'pandas.DataFrame'>
```

**Common mistakes:** (1) The file must be in the same folder as your notebook, or you must give the full path. (2) `read_excel` and `read_parquet` raise an `ImportError` if `openpyxl` / `pyarrow` is not installed. Install them with `pip install openpyxl pyarrow`.

# 3. Inspecting a DataFrame

These methods help you understand a new dataset quickly. `df` below is the 20-row student table.

```
df = pd.read_csv('student_data.csv')
```

## 3.1 head() and tail()

`head()` shows the first rows and `tail()` shows the last rows. The default is 5 rows. Put a number inside the brackets to choose how many.

SYNTAX

```
df.head(n)    # first n rows
df.tail(n)    # last n rows
```

```
# first 4 rows
df.head(4)
```

OUTPUT

```
  StudentID          FullName  ...  Instructor    Location
0    PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
1    PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
2    PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
3    PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet

[4 rows x 9 columns]
```

```
# last 5 rows (default)
df.tail()
```

OUTPUT

```
   StudentID      FullName  ...  Instructor    Location
15    PH1016  Ziaur Rahman  ...   Ms. Salma  Chattogram
16    PH1017   Afsana Mimi  ...   Mr. Karim       Dhaka
17    PH1018   Babul Ahmed  ...   Ms. Salma      Sylhet
18    PH1019  Faria Rahman  ...   Mr. David  Chattogram
19    PH1020    Nasir Khan  ...   Ms. Salma       Dhaka

[5 rows x 9 columns]
```

> **Note:** `head()` and `tail()` are DataFrame methods, so they need brackets and are called as `df.head()`.

## 3.2 Columns and index

`df.columns` holds the column names and `df.index` holds the row labels. Converting them to a NumPy array lets you check their data type.

```
# the column names
df.columns
```

OUTPUT

```
Index(['StudentID', 'FullName', 'Data Structure Marks', 'Algorithm Marks', 'Python Marks',
       'CompletionStatus', 'EnrollmentDate', 'Instructor', 'Location'],
      dtype='str')
```

```
import numpy as np

# the columns as a NumPy array
col = np.array(df.columns)

print(col.dtype)
```

OUTPUT

```
object
```

```
# the row labels
df.index
```

OUTPUT

```
RangeIndex(start=0, stop=20, step=1)
```

```
# the index as a NumPy array
ind = np.array(df.index)

print(ind)
print(ind.dtype)
```

OUTPUT

```
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
int64
```

> **Note:** The column names are text, so NumPy stores them with the type `object`. The index holds whole numbers, so it is `int64`.

## 3.3 info()

`info()` prints a summary: number of rows, each column's name, non-null count and data type, and memory used. It is the quickest way to spot missing values.

```
df.info()
```

OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 9 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   StudentID             20 non-null     str
 1   FullName              20 non-null     str
 2   Data Structure Marks  16 non-null     float64
 3   Algorithm Marks       16 non-null     float64
 4   Python Marks          15 non-null     float64
 5   CompletionStatus      20 non-null     str
 6   EnrollmentDate        20 non-null     str
 7   Instructor            20 non-null     str
 8   Location              20 non-null     str
dtypes: float64(3), str(6)
memory usage: 2.6 KB
```

> **Note:** Here `Data Structure Marks` has 16 non-null values out of 20, so 4 values are missing. Text columns show as `object` in older pandas and as `str` in pandas 3.

## 3.4 sample()

`sample(n)` shows `n` random rows. You get different rows every time you run it.

```
df.sample(10)
```

OUTPUT

```
   StudentID          FullName  ...  Instructor    Location
3     PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
16    PH1017       Afsana Mimi  ...   Mr. Karim       Dhaka
6     PH1007    Mahmudul Hasan  ...   Mr. Karim       Dhaka
10    PH1011      Rahim Sheikh  ...   Mr. Karim      Khulna
2     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
14    PH1015      Wahiduzzaman  ...   Mr. Karim       Dhaka
4     PH1005       Kamal Uddin  ...   Mr. Karim  Chattogram
17    PH1018       Babul Ahmed  ...   Ms. Salma      Sylhet
7     PH1008       Nadia Islam  ...   Ms. Salma  Chattogram
1     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram

[10 rows x 9 columns]
```

> **Note:** To get the same random rows every time, add `random_state`, for example `df.sample(10, random_state=1)`.

## 3.5 describe()

`describe()` gives statistics for the numeric columns only (columns where calculations are possible).

```
df.describe()
```

OUTPUT

```
       Data Structure Marks  Algorithm Marks  Python Marks
count             16.000000        16.000000     15.000000
mean              84.000000        84.000000     85.666667
std                6.501282         6.501282      5.394000
min               72.000000        72.000000     76.000000
25%               79.500000        79.500000     83.000000
50%               85.500000        85.500000     85.000000
75%               88.250000        88.250000     88.500000
max               94.000000        94.000000     95.000000
```

> **Note:** `count` is the number of non-null values, so it is 16 for the marks columns (4 students have no marks). `25%`, `50%` and `75%` are the quartiles; `50%` is the median.

## 3.6 Selecting columns by data type

`select_dtypes()` picks columns by their data type. Use `include=` to keep a type and `exclude=` to leave one out. `'number'` means all numeric columns (whole numbers and decimals).

SYNTAX

```
df.select_dtypes(include='number')    # only the numeric columns
df.select_dtypes(exclude='number')    # every column that is not numeric
```

```
num_col = df.select_dtypes(include='number').columns
str_col = df.select_dtypes(exclude='number').columns

print(num_col)
print(str_col)
```

OUTPUT

```
Index(['Data Structure Marks', 'Algorithm Marks', 'Python Marks'], dtype='str')
Index(['StudentID', 'FullName', 'CompletionStatus', 'EnrollmentDate', 'Instructor',
       'Location'],
      dtype='str')
```

> **Note:** `num_col` and `str_col` hold only the **column names**. To get the data, use them inside square brackets: `df[num_col]` or `df[str_col]`.

```
# the data of the numeric columns
df[num_col].head(3)
```

OUTPUT

```
   Data Structure Marks  Algorithm Marks  Python Marks
0                  85.0             85.0          88.0
1                  92.0             92.0           NaN
2                  88.0             88.0          85.0
```

`select_dtypes()` chooses columns only. To choose rows as well, use the names as the column part of `df.loc[rows, columns]` (see Section 5).

```
# rows 0 to 4, numeric columns only
df.loc[0:4, num_col]
```

OUTPUT

```
   Data Structure Marks  Algorithm Marks  Python Marks
0                  85.0             85.0          88.0
1                  92.0             92.0           NaN
2                  88.0             88.0          85.0
3                  78.0             78.0          82.0
4                   NaN              NaN          95.0
```

```
# rows 2, 3 and 19, text columns only
df.loc[[2, 3, 19], str_col]
```

OUTPUT

```
   StudentID          FullName  ... Instructor Location
2     PH1003     Imran Hossain  ...  Mr. Karim    Dhaka
3     PH1004  Jannatul Ferdous  ...  Ms. Salma   Sylhet
19    PH1020        Nasir Khan  ...  Ms. Salma    Dhaka

[3 rows x 6 columns]
```

```
# only the completed students (a condition), numeric columns only
df.loc[df['CompletionStatus'] == 'Completed', num_col]
```

OUTPUT

```
    Data Structure Marks  Algorithm Marks  Python Marks
0                  85.0             85.0          88.0
2                  88.0             88.0          85.0
3                  78.0             78.0          82.0
5                  75.0             75.0          78.0
..                  ...              ...           ...
14                 86.0             86.0          84.0
16                 90.0             90.0          93.0
17                 88.0             88.0          85.0
19                 86.0             86.0          89.0

[13 rows x 3 columns]
```

**Note:** `exclude='number'` means “everything that is not numeric”. A date column that is still text, such as `EnrollmentDate`, is therefore included in `str_col` until you convert it with `pd.to_datetime()` (Section 17).

## 4. Creating a DataFrame from Python Data

Use `pd.DataFrame(data)` to build a DataFrame from lists, tuples or dictionaries. `columns=` sets the column names and `index=` sets the row labels.

SYNTAX

```
pd.DataFrame(data, columns=[...], index=[...])
```

### 4.1 From a list of lists

```
my_list = [['Alice', 25], ['Bob', 30], ['Charlie', 28]]

list_df = pd.DataFrame(my_list, columns=['Name', 'Age'], index=[1, 2, 3])

print(type(list_df))
list_df
```

OUTPUT

```
<class 'pandas.DataFrame'>
      Name  Age
1    Alice   25
2      Bob   30
3  Charlie   28
```

### 4.2 From a tuple

```
my_tuple = (('Alice', 25), ('Bob', 30), ('Charlie', 28))

tuple_df = pd.DataFrame(my_tuple, columns=['Name', 'Age'])

tuple_df
```

OUTPUT

```
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   28
```

No `index=` was given here, so pandas uses the default index 0, 1, 2.

### 4.3 From a dictionary

Each **key** becomes a column name and each **list** becomes the values of that column.

```
my_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 29, 28]
}

dict_df = pd.DataFrame(my_dict)

dict_df
```

OUTPUT

```
      Name  Age
0    Alice   25
1      Bob   29
2  Charlie   28
```

> **Common mistake:** all lists in the dictionary must have the same length, otherwise pandas raises `ValueError: All arrays must be of the same length`.

### 4.4 From a list of dictionaries

Each dictionary is one **row**. If a key is missing in a row, pandas fills that cell with `NaN`.

```
my_data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
    {'Name': 'Bob', 'Age': 30, 'City': 'Paris'},
    {'Name': 'Charlie', 'Age': 28}
]

data_df = pd.DataFrame(my_data)

data_df
```

OUTPUT

```
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   28       NaN
```

```
print(type(data_df['Name']))
```

OUTPUT

```
<class 'pandas.Series'>
```

## 5. Selecting Data

Use `df['column']` to pick columns and `df.loc[rows, columns]` to pick rows and columns by their **labels** (the index values and column names).

SYNTAX

```
df['column']                      # one column  -> Series
df[['col1', 'col2']]              # many columns -> DataFrame
df.loc[row_label]                 # one row     -> Series
df.loc[row_start:row_end, col]    # rows and columns by label
```

> **Correction:** the notes had the pattern `df.loc[ind start:ind end, col_start, col_end]`. The correct pattern is `df.loc[rows, columns]`: the row part and the column part are separated by **one** comma.

### 5.1 One column

```
# accessing a column
print(df['FullName'])
print(type(df['FullName']))
```

OUTPUT

```
0          Alif Rahman
1        Fatima Akhter
2        Imran Hossain
3     Jannatul Ferdous
            ...
16         Afsana Mimi
17         Babul Ahmed
18        Faria Rahman
19          Nasir Khan
Name: FullName, Length: 20, dtype: str
<class 'pandas.Series'>
```

## 5.2 Several columns

Use a **list** inside the brackets (double square brackets). The result is a DataFrame.

```
df[['FullName', 'StudentID']]
```

OUTPUT

```
            FullName StudentID
0        Alif Rahman    PH1001
1      Fatima Akhter    PH1002
2      Imran Hossain    PH1003
3   Jannatul Ferdous    PH1004
..               ...       ...
16       Afsana Mimi    PH1017
17       Babul Ahmed    PH1018
18      Faria Rahman    PH1019
19        Nasir Khan    PH1020

[20 rows x 2 columns]
```

**Common mistake:** `df['FullName', 'StudentID']` (without the inner list) gives a `KeyError`.

## 5.3 One row

```
print(df.loc[0])
print(type(df.loc[0]))
```

OUTPUT

```
StudentID                   PH1001
FullName                Alif Rahman
Data Structure Marks          85.0
Algorithm Marks               85.0
Python Marks                  88.0
CompletionStatus         Completed
EnrollmentDate          2024-01-15
Instructor               Mr. Karim
Location                     Dhaka
Name: 0, dtype: object
<class 'pandas.Series'>
```

## 5.4 Several rows

```
# multiple rows (normal list)
df.loc[[2, 3, 19]]
```

OUTPUT

```
   StudentID          FullName  ...  Instructor  Location
2     PH1003     Imran Hossain  ...   Mr. Karim     Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma     Sylhet
19    PH1020        Nasir Khan  ...   Ms. Salma     Dhaka

[3 rows x 9 columns]
```

```
# multiple rows (range)
df.loc[3:7]
```

OUTPUT

```
  StudentID          FullName  ...  Instructor    Location
3    PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
4    PH1005       Kamal Uddin  ...   Mr. Karim  Chattogram
5    PH1006       Laila Begum  ...   Ms. Salma    Rajshahi
6    PH1007    Mahmudul Hasan  ...   Mr. Karim       Dhaka
7    PH1008       Nadia Islam  ...   Ms. Salma  Chattogram

[5 rows x 9 columns]
```

**Important:** with `.loc`, a range like `3:7` **includes** the end label, so you get rows 3, 4, 5, 6 and 7.

## 5.5 Columns with .loc

A colon `:` means “all rows”.

```
# single column
df.loc[:, 'Python Marks']
```

OUTPUT

```
0     88.0
1      NaN
2     85.0
3     82.0
      ...
16    93.0
17    85.0
18     NaN
19    89.0
Name: Python Marks, Length: 20, dtype: float64
```

```
# multiple columns
print(df.loc[:, ['Python Marks', 'Algorithm Marks']])
print(type(df.loc[:, ['Python Marks', 'Algorithm Marks']]))
```

OUTPUT

```
    Python Marks  Algorithm Marks
0           88.0             85.0
1            NaN             92.0
2           85.0             88.0
3           82.0             78.0
..           ...              ...
16          93.0             90.0
17          85.0             88.0
18           NaN              NaN
19          89.0             86.0

[20 rows x 2 columns]
<class 'pandas.DataFrame'>
```

## 5.6 Rows and columns together

```
# rows 3 to 7, one column
df.loc[3:7, 'CompletionStatus']
```

OUTPUT

```
3      Completed
4    In Progress
5      Completed
6    In Progress
7      Completed
Name: CompletionStatus, dtype: str
```

# 6. Index, iloc and Renaming Columns

## 6.1 set_index()

`set_index('column')` makes that column the new row labels (index).

```
# the values of the StudentID column become the new index
# it returns a NEW DataFrame
df_index = df.set_index('StudentID')

df_index
```

OUTPUT

```
                FullName  Data Structure Marks  ...  Instructor    Location
StudentID                                       ...
PH1001       Alif Rahman                  85.0  ...   Mr. Karim       Dhaka
PH1002      Fatima Akhter                 92.0  ...   Ms. Salma  Chattogram
PH1003      Imran Hossain                 88.0  ...   Mr. Karim       Dhaka
PH1004   Jannatul Ferdous                 78.0  ...   Ms. Salma      Sylhet
...                   ...                  ...  ...         ...         ...
PH1017       Afsana Mimi                  90.0  ...   Mr. Karim       Dhaka
PH1018       Babul Ahmed                  88.0  ...   Ms. Salma      Sylhet
PH1019      Faria Rahman                   NaN  ...   Mr. David  Chattogram
PH1020        Nasir Khan                 86.0  ...   Ms. Salma       Dhaka

[20 rows x 8 columns]
```

```
# to change the original DataFrame itself, use inplace=True
# df.set_index('StudentID', inplace=True)

# .loc now needs the new labels (not 3 and 7)
df_index.loc['PH1004':'PH1008']
```

OUTPUT

```
                FullName  Data Structure Marks  ...  Instructor    Location
StudentID                                       ...
PH1004   Jannatul Ferdous                 78.0  ...   Ms. Salma      Sylhet
PH1005        Kamal Uddin                  NaN  ...   Mr. Karim  Chattogram
PH1006        Laila Begum                 75.0  ...   Ms. Salma    Rajshahi
PH1007     Mahmudul Hasan                 80.0  ...   Mr. Karim       Dhaka
PH1008        Nadia Islam                 81.0  ...   Ms. Salma  Chattogram

[5 rows x 8 columns]
```

> **Common mistake:** after `set_index('StudentID')`, `df_index.loc[3:7]` raises a `TypeError` because the labels are now `'PH1001'`, `'PH1002'`, ... and not numbers. Use the new labels, or use `.iloc` (next topic).

## 6.2 iloc (position based)

`iloc` selects by **position number** (0, 1, 2, ...) for both rows and columns. It does not care what the labels are.

SYNTAX

```
df.iloc[row_start:row_end, col_start:col_end]
```

```
# index doesn't matter, only position: all rows, columns 0 to 4
df_index.iloc[:, 0:5]
```

OUTPUT

```
                  FullName  Data Structure Marks  ...  Python Marks  CompletionStatus
StudentID                                         ...
PH1001         Alif Rahman                  85.0  ...          88.0         Completed
PH1002       Fatima Akhter                  92.0  ...           NaN       In Progress
PH1003       Imran Hossain                  88.0  ...          85.0         Completed
PH1004    Jannatul Ferdous                  78.0  ...          82.0         Completed
...                    ...                   ...  ...           ...               ...
PH1017         Afsana Mimi                  90.0  ...          93.0         Completed
PH1018         Babul Ahmed                  88.0  ...          85.0         Completed
PH1019        Faria Rahman                   NaN  ...           NaN       Not Started
PH1020          Nasir Khan                  86.0  ...          89.0         Completed

[20 rows x 5 columns]
```

| | .loc | .iloc |
|---|---|---|
| Uses | labels (index values, column names) | positions (0, 1, 2, ...) |
| End of a slice | included | excluded |
| Example | df.loc[3:7] gives rows 3 to 7 | df.iloc[3:7] gives rows 3 to 6 |

## 6.3 Renaming columns

`rename(columns={old_name: new_name})` changes column names. Give a dictionary for the columns you want to change.

SYNTAX

```
df.rename(columns={'old_name': 'new_name'}, inplace=True)
```

```
df.rename(columns={'FullName': 'Full Name',
                   'Algorithm Marks': 'Algo Marks'}, inplace=True)

df.columns
```

OUTPUT

```
Index(['StudentID', 'Full Name', 'Data Structure Marks', 'Algo Marks', 'Python Marks',
       'CompletionStatus', 'EnrollmentDate', 'Instructor', 'Location'],
      dtype='str')
```

**Common mistake:** with `inplace=True` the method changes `df` and returns `None`. Never write `df = df.rename(..., inplace=True)`, because `df` becomes `None`.

**Note:** From here until Section 10.3, the examples continue with this same `df` (columns renamed as above). Section 10.3 starts a new small DataFrame.

# 7. Changing Data

## 7.1 Deleting a row

`drop(label)` removes the row with that index label.

```
# row deletion
df.drop(0, inplace=True)

df
```

OUTPUT

```
   StudentID         Full Name  ...  Instructor    Location
1     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
2     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
4     PH1005       Kamal Uddin  ...   Mr. Karim  Chattogram
..       ...               ...  ...         ...         ...
16    PH1017       Afsana Mimi  ...   Mr. Karim       Dhaka
17    PH1018       Babul Ahmed  ...   Ms. Salma      Sylhet
18    PH1019      Faria Rahman  ...   Mr. David  Chattogram
19    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

[19 rows x 9 columns]
```

## 7.2 Deleting a column

`axis=1` tells pandas that the name is a **column**. (`axis=0`, the default, means rows.)

```
# column deletion
# axis=1 -> delete the full column
df.drop('Instructor', axis=1, inplace=True)

df.columns
```

OUTPUT

```
Index(['StudentID', 'Full Name', 'Data Structure Marks', 'Algo Marks', 'Python Marks',
       'CompletionStatus', 'EnrollmentDate', 'Location'],
      dtype='str')
```

> **Common mistake:** `df.drop('Instructor')` without `axis=1` looks for a **row** named `'Instructor'` and raises a `KeyError`.

## 7.3 Changing one value

SYNTAX

```
df.loc[row_label, 'column'] = new_value
```

```
df.loc[1, 'Python Marks'] = 90

df.loc[1, 'CompletionStatus'] = 'Completed'

df.loc[1]
```

OUTPUT

```
StudentID                    PH1002
Full Name              Fatima Akhter
Data Structure Marks            92.0
Algo Marks                      92.0
Python Marks                    90.0
CompletionStatus           Completed
EnrollmentDate            2024-01-20
Location                  Chattogram
Name: 1, dtype: object
```

### 7.4 Changing many values

```
# add 2 to Python Marks for rows 1 to 3
df.loc[1:3, 'Python Marks'] += 2

df.loc[1:3, 'Python Marks']
```

OUTPUT

```
1    92.0
2    87.0
3    84.0
Name: Python Marks, dtype: float64
```

> **Note:** `+=` works on the whole selected range at once. Rows 1, 2 and 3 are all included because `.loc` includes the end label.

## 8. Looping Through Rows

### 8.1 iterrows()

`iterrows()` gives two things for each row: the index (`i`) and the full row as a Series (`series`).

```
# i -> index, series -> the full row
for i, series in df.iterrows():
    print(f"{i} :\n{series}")
```

OUTPUT

```
1 :
StudentID                    PH1002
Full Name              Fatima Akhter
Data Structure Marks            92.0
Algo Marks                      92.0
Python Marks                    92.0
CompletionStatus           Completed
EnrollmentDate            2024-01-20
Location                  Chattogram
Name: 1, dtype: object
2 :
StudentID                    PH1003
Full Name              Imran Hossain
Data Structure Marks            88.0
Algo Marks                      88.0
Python Marks                    87.0
CompletionStatus           Completed
EnrollmentDate            2024-02-10
Location                       Dhaka
Name: 2, dtype: object
...
```

*(Output shortened: the loop prints all 19 remaining rows.)*

### 8.2 itertuples()

`itertuples()` gives each row as a **named tuple**. `index=False` leaves out the index.

```
# each row comes as a tuple instead of a Series
for i in df.itertuples(index=False):
    print(i)
```

OUTPUT

```
Pandas(StudentID='PH1002', _1='Fatima Akhter', _2=92.0, _3=92.0, _4=92.0,
    CompletionStatus='Completed', EnrollmentDate='2024-01-20', Location='Chattogram')
Pandas(StudentID='PH1003', _1='Imran Hossain', _2=88.0, _3=88.0, _4=87.0,
    CompletionStatus='Completed', EnrollmentDate='2024-02-10', Location='Dhaka')
Pandas(StudentID='PH1004', _1='Jannatul Ferdous', _2=78.0, _3=78.0, _4=84.0,
    CompletionStatus='Completed', EnrollmentDate='2024-02-12', Location='Sylhet')
...
```

*(Output shortened: one tuple is printed for each remaining row.)*

> **Note:** A column name with spaces cannot be used as a tuple field name, so pandas renames it to `_1`, `_2`, ... (position numbers). That is why `Full Name`, `Data Structure Marks`, `Algo Marks` and `Python Marks` appear as `_1` to `_4`. For a name without spaces you can read a single field, for example `i.StudentID`.
>
> **Note:** Loops over rows are slow on large data. For big tables, use column operations (Sections 11, 15 and 16).

## 9. Sorting

`sort_values()` sorts the rows by one or more columns. It returns a **new** DataFrame and the original `df` does not change. `copy` below is just a variable name.

SYNTAX

```
df.sort_values(by, ascending=True)
```

```
# ascending order (small to big) is the default
copy = df.sort_values('Data Structure Marks')

copy[['Full Name', 'Data Structure Marks', 'Python Marks']]
```

OUTPUT

```
           Full Name  Data Structure Marks  Python Marks
8         Omar Faruq                  72.0          76.0
5        Laila Begum                  75.0          78.0
12      Tanvir Ahmed                  75.0          79.0
3   Jannatul Ferdous                  78.0          84.0
..               ...                   ...           ...
4        Kamal Uddin                   NaN          95.0
10      Rahim Sheikh                   NaN          91.0
13        Urmi Akter                   NaN           NaN
18      Faria Rahman                   NaN           NaN

[19 rows x 3 columns]
```

```
# descending sort
copy = df.sort_values('Data Structure Marks', ascending=False)

copy[['Full Name', 'Data Structure Marks', 'Python Marks']]
```

OUTPUT

```
        Full Name  Data Structure Marks  Python Marks
15   Ziaur Rahman                  94.0           NaN
1   Fatima Akhter                  92.0          92.0
16    Afsana Mimi                  90.0          93.0
9    Priya Sharma                  89.0          88.0
..            ...                   ...           ...
4     Kamal Uddin                   NaN          95.0
10   Rahim Sheikh                   NaN          91.0
13     Urmi Akter                   NaN           NaN
18   Faria Rahman                   NaN           NaN

[19 rows x 3 columns]
```

## 9.1 Sorting by more than one column

Give a **list** of columns. If two rows have the same value in the first column, the second column decides their order. (`head(10)` shows the first 10 rows so you can see the ties.)

```
# if Data Structure Marks are the same, Python Marks decides the order
copy = df.sort_values(['Data Structure Marks', 'Python Marks'], ascending=False)

copy[['Full Name', 'Data Structure Marks', 'Python Marks']].head(10)
```

OUTPUT

```
          Full Name  Data Structure Marks  Python Marks
15     Ziaur Rahman                  94.0           NaN
1     Fatima Akhter                  92.0          92.0
16      Afsana Mimi                  90.0          93.0
9      Priya Sharma                  89.0          88.0
2     Imran Hossain                  88.0          87.0
17      Babul Ahmed                  88.0          85.0
19       Nasir Khan                  86.0          89.0
14     Wahiduzzaman                  86.0          84.0
11  Sadia Chowdhury                  85.0          87.0
7       Nadia Islam                  81.0          85.0
```

To use a different order for each column, give `ascending` a list with one value per column.

```
# Data Structure Marks: big to small, Python Marks: small to big
copy = df.sort_values(['Data Structure Marks', 'Python Marks'], ascending=[False, True])

copy[['Full Name', 'Data Structure Marks', 'Python Marks']].head(10)
```

OUTPUT

```
          Full Name  Data Structure Marks  Python Marks
15     Ziaur Rahman                  94.0           NaN
1     Fatima Akhter                  92.0          92.0
16      Afsana Mimi                  90.0          93.0
9      Priya Sharma                  89.0          88.0
17      Babul Ahmed                  88.0          85.0
2     Imran Hossain                  88.0          87.0
14     Wahiduzzaman                  86.0          84.0
19       Nasir Khan                  86.0          89.0
11  Sadia Chowdhury                  85.0          87.0
7       Nadia Islam                  81.0          85.0
```

**Correction:** the notes used `ascending=[0, 1]`. This works, but `[False, True]` is clearer and is the recommended way.

**Note:** Missing values (`NaN`) are placed at the **end** by default, in both ascending and descending sorts.

# 10. Filtering Rows

Filtering keeps only the rows that match a condition. A condition such as `df['CompletionStatus'] == 'Completed'` gives `True` or `False` for every row, and `df.loc[condition]` keeps the `True` rows.

SYNTAX

```
df.loc[condition]
```

## 10.1 One condition

```
# filter with a condition
not_started = df.loc[df['CompletionStatus'] == 'Not Started']

not_started[['StudentID', 'Full Name', 'CompletionStatus']]
```

OUTPUT

```
   StudentID     Full Name CompletionStatus
13    PH1014    Urmi Akter      Not Started
18    PH1019  Faria Rahman      Not Started
```

```
completed = df.loc[df['CompletionStatus'] == 'Completed']

completed[['StudentID', 'Full Name', 'CompletionStatus']]
```

OUTPUT

```
   StudentID         Full Name CompletionStatus
1     PH1002     Fatima Akhter        Completed
2     PH1003     Imran Hossain        Completed
3     PH1004  Jannatul Ferdous        Completed
5     PH1006       Laila Begum        Completed
..       ...               ...              ...
14    PH1015      Wahiduzzaman        Completed
16    PH1017       Afsana Mimi        Completed
17    PH1018       Babul Ahmed        Completed
19    PH1020        Nasir Khan        Completed

[13 rows x 3 columns]
```

**Common mistake:** use `==` to compare. A single `=` assigns a value and gives a syntax error inside a condition.

## 10.2 More than one condition

Use `&` for **and**, and put each condition inside its own brackets.

```
# completed AND data structure marks greater than or equal to 90
completed_ds90 = df.loc[(df['CompletionStatus'] == 'Completed') &
                        (df['Data Structure Marks'] >= 90)]

completed_ds90[['Full Name', 'CompletionStatus', 'Data Structure Marks']]
```

OUTPUT

```
        Full Name CompletionStatus  Data Structure Marks
1   Fatima Akhter        Completed                  92.0
16    Afsana Mimi        Completed                  90.0
```

**Common mistake:** do not use the words `and` / `or` between conditions; pandas raises `ValueError: The truth value of a Series is ambiguous`. Use `&` (and) and `|` (or) with brackets around each condition.

## 10.3 Filtering text with str.contains()

`str.contains()` checks every text value in a column and returns `True` when the text matches. The pattern is a **regular expression**, so special symbols like `^`, `$` and `|` work.

SYNTAX

```
df.loc[df['column'].str.contains(pattern, case=True)]
```

This example uses a new small DataFrame called `df`:

```
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eve", "Frank", "Grace", "Hannah",
             "Sakib"],
    "City": ["New York", "Los Angeles", "Newark", "Boston", "New Delhi", "Chicago",
             "New Orleans", "Houston", "H Los Ang"],
    "Department": ["HR", "IT", "Finance", "IT", "HR", "Marketing", "Finance", "HR",
                   "HR"],
    "Salary": [50000, 60000, 55000, 70000, 52000, 58000, 62000, 51000, 70000]
}

df = pd.DataFrame(data)

df
```

OUTPUT

```
      Name         City Department  Salary
0    Alice     New York         HR   50000
1      Bob  Los Angeles         IT   60000
2  Charlie       Newark    Finance   55000
3    David       Boston         IT   70000
4      Eve    New Delhi         HR   52000
5    Frank      Chicago  Marketing   58000
6    Grace  New Orleans    Finance   62000
7   Hannah      Houston         HR   51000
8    Sakib    H Los Ang         HR   70000
```

```
# City contains the text "New"
df.loc[df['City'].str.contains("New")]
```

OUTPUT

```
      Name         City Department  Salary
0    Alice     New York         HR   50000
2  Charlie       Newark    Finance   55000
4      Eve    New Delhi         HR   52000
6    Grace  New Orleans    Finance   62000
```

```
# handle case sensitivity: case=False ignores upper/lower case
df.loc[df['City'].str.contains("new", case=False)]
```

OUTPUT

```
      Name         City Department  Salary
0    Alice     New York         HR   50000
2  Charlie       Newark    Finance   55000
4      Eve    New Delhi         HR   52000
6    Grace  New Orleans    Finance   62000
```

```
# starts with Los
df.loc[df['City'].str.contains(r"^Los")]
```

OUTPUT

```
  Name         City Department  Salary
1  Bob  Los Angeles         IT   60000
```

```
# ends with rk
df.loc[df['City'].str.contains(r"rk$")]
```

OUTPUT

```
      Name      City Department  Salary
0    Alice  New York         HR   50000
2  Charlie    Newark    Finance   55000
```

```
# name starts with a vowel
df.loc[df['Name'].str.contains(r"^[AEIOU]")]
```

OUTPUT

```
    Name       City Department  Salary
0  Alice   New York         HR   50000
4    Eve  New Delhi         HR   52000
```

```
# contains New or Los
df.loc[df['City'].str.contains(r"New|Los")]
```

OUTPUT

```
      Name         City Department  Salary
0    Alice     New York         HR   50000
1      Bob  Los Angeles         IT   60000
2  Charlie       Newark    Finance   55000
4      Eve    New Delhi         HR   52000
6    Grace  New Orleans    Finance   62000
8    Sakib    H Los Ang         HR   70000
```

| Symbol | Meaning | Example |
|---|---|---|
| ^ | start of the text | ^Los: starts with Los |
| $ | end of the text | rk$: ends with rk |
| [AEIOU] | any one of these letters | ^[AEIOU]: starts with a vowel |
| \| | or | New\|Los: contains New or Los |

**Note:** Without `^` or `$`, `contains` matches the text **anywhere** in the value. That is why “New” also matches “Newark”. The pattern is case-sensitive unless you add `case=False`, so `^[AEIOU]` matches only capital vowels. Start patterns with `r` (for example `r"^Los"`) to write them as raw strings.

# 11. Adding New Columns

Write `df['New Column'] = values` to create a column. If the name already exists, the old values are replaced.

From here the examples start from a fresh `df`. In these notes `student_data.csv` has one extra completely empty row at the end (row 20). It is used later to practise missing values.

```
df = pd.read_csv('student_data.csv')

df[['StudentID', 'FullName', 'Python Marks']].tail(3)
```

OUTPUT

```
   StudentID      FullName  Python Marks
18    PH1019  Faria Rahman           NaN
19    PH1020    Nasir Khan          89.0
20       NaN           NaN           NaN
```

## 11.1 A column with one constant value

```
# adding a column with a constant value
df['Country'] = 'Bangladesh'

df[['FullName', 'Country']].head(3)
```

OUTPUT

```
        FullName     Country
0    Alif Rahman  Bangladesh
1  Fatima Akhter  Bangladesh
2  Imran Hossain  Bangladesh
```

## 11.2 A column calculated from other columns

```
# adding the values of 3 columns
df['Total Marks'] = (df['Data Structure Marks'] + df['Python Marks']
                     + df['Algorithm Marks'])

df[['FullName', 'Total Marks']].head(6)
```

OUTPUT

```
           FullName  Total Marks
0       Alif Rahman        258.0
1     Fatima Akhter          NaN
2     Imran Hossain        261.0
3  Jannatul Ferdous        238.0
4       Kamal Uddin          NaN
5       Laila Begum        228.0
```

**Note:** If any one of the three marks is missing, the total is `NaN`. Fatima (row 1) has no Python mark, so her total is `NaN`. (Section 15 shows `sum(axis=1)`, which skips missing values.)

## 11.3 A column from a condition: np.where()

`np.where(condition, value_if_true, value_if_false)` fills the column using a condition. It needs `import numpy as np`.

SYNTAX

```
np.where(condition, 'value if True', 'value if False')
```

```
import numpy as np

# students with marks above 90 get A+, everyone else gets A
df['A+ in DS'] = np.where(df['Data Structure Marks'] > 90, 'A+', 'A')

df[['FullName', 'Data Structure Marks', 'A+ in DS']].head(6)
```

OUTPUT

```
           FullName  Data Structure Marks A+ in DS
0       Alif Rahman                  85.0        A
1     Fatima Akhter                  92.0       A+
2     Imran Hossain                  88.0        A
3  Jannatul Ferdous                  78.0        A
4       Kamal Uddin                   NaN        A
5       Laila Begum                  75.0        A
```

**Watch out:** a missing mark is not greater than 90, so `NaN` rows also get `'A'` (see Kamal Uddin above). `np.where` does not leave them empty.

## 11.4 A True/False column

```
# the condition itself creates a True/False column
df['Passed in DS'] = df['Data Structure Marks'] > 70

df[['FullName', 'Data Structure Marks', 'Passed in DS']].head(6)
```

OUTPUT

```
           FullName  Data Structure Marks  Passed in DS
0       Alif Rahman                  85.0          True
1     Fatima Akhter                  92.0          True
2     Imran Hossain                  88.0          True
3  Jannatul Ferdous                  78.0          True
4       Kamal Uddin                   NaN         False
5       Laila Begum                  75.0          True
```

### 11.5 A column from text

`str.split(' ')` splits each full name at the spaces into a list of words, and `.str[0]` takes the first word.

```
# split the full name and use the first part to create a new column
df['First Name'] = df['FullName'].str.split(' ').str[0]

df[['FullName', 'First Name']].head(4)
```

OUTPUT

```
           FullName First Name
0       Alif Rahman       Alif
1     Fatima Akhter     Fatima
2     Imran Hossain      Imran
3  Jannatul Ferdous   Jannatul
```

### 11.6 Saving a DataFrame to a CSV file

Changes to `df` stay in memory only. The original CSV file is **not** changed. To keep the new columns, save a new file with `to_csv()`.

```
# the file we imported from is not changed when df changes
df1 = pd.read_csv('student_data.csv')
print(df1.columns.tolist())
```

OUTPUT

```
['StudentID', 'FullName', 'Data Structure Marks', 'Algorithm Marks', 'Python Marks',
    'CompletionStatus', 'EnrollmentDate', 'Instructor', 'Location']
```

```
# create a new CSV file from df
df.to_csv('new_data.csv', index=False)

print(pd.read_csv('new_data.csv').columns.tolist())
```

OUTPUT

```
['StudentID', 'FullName', 'Data Structure Marks', 'Algorithm Marks', 'Python Marks',
    'CompletionStatus', 'EnrollmentDate', 'Instructor', 'Location', 'Country', 'Total Marks',
    'A+ in DS', 'Passed in DS', 'First Name']
```

> **Correction:** the notes used `df.to_csv('new_data.csv')`. That also writes the index as an extra first column (it comes back as `Unnamed: 0` when you read the file). Add `index=False` to avoid it.

## 12. Unique Values: unique() and nunique()

`unique()` lists the different values of a column. `nunique()` counts them.

```
data = {
    "Name": ["Alice", "Bob", "Charlie", "Alice", "David", "Bob"],
    "City": ["New York", "London", "Paris", "New York", "Tokyo", "London"],
    "Score": [85, 90, 78, 85, 95, 90]
}

df = pd.DataFrame(data)

df
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     85
4    David     Tokyo     95
5      Bob    London     90
```

```
# unique works only on a Series (one column)
print(df['Name'].unique())

print(len(df['Name'].unique()))
```

OUTPUT

```
<ArrowStringArray>
['Alice', 'Bob', 'Charlie', 'David']
Length: 4, dtype: str
4
```

**Note:** In older pandas versions `unique()` prints as `array(['Alice', 'Bob', 'Charlie', 'David'], dtype=object)`. It is the same list of values.

**Common mistake:** `df.unique()` raises an `AttributeError`. `unique()` works only on one column (a Series).

### 12.1 Missing values in unique() and nunique()

```
df1 = pd.read_csv('student_data.csv')

# unique() keeps NaN and returns all the values
print(df1['Data Structure Marks'].unique())

# nunique() ignores NaN and counts only the real values
print(df1['Data Structure Marks'].nunique())
```

OUTPUT

```
[85. 92. 88. 78. nan 75. 80. 81. 72. 89. 86. 94. 90.]
12
```

### 12.2 nunique() on a whole DataFrame

On a DataFrame, `nunique()` returns the number of different values in **each column**.

```
df.nunique()
```

OUTPUT

```
Name     4
City     4
Score    4
dtype: int64
```

## 13. Missing (Null) Values

Missing values appear as `NaN` (not a number). Pandas can find them, remove them or fill them. `df1` below is `student_data.csv` read again.

### 13.1 Finding missing values

| Method | What it gives |
|---|---|
| `df.isnull()` | True/False table for every cell: True where a value is missing |
| `series.isnull()` | True for each missing value of one column |
| `series.notnull()` | the opposite: True where a value exists |
| `series.hasnans` | one True/False answer: True if the column has at least one missing value |

```
df1.isnull()
```

OUTPUT

```
    StudentID  FullName  ...  Instructor  Location
0       False     False  ...       False     False
1       False     False  ...       False     False
2       False     False  ...       False     False
3       False     False  ...       False     False
..        ...       ...  ...         ...       ...
17      False     False  ...       False     False
18      False     False  ...       False     False
19      False     False  ...       False     False
20       True      True  ...        True      True

[21 rows x 9 columns]
```

```
# the null values show as True
df1['Data Structure Marks'].isnull()
```

OUTPUT

```
0     False
1     False
2     False
3     False
      ...
17    False
18     True
19    False
20     True
Name: Data Structure Marks, Length: 21, dtype: bool
```

```
# the values that are not null show as True
df1['Data Structure Marks'].notnull()
```

OUTPUT

```
0      True
1      True
2      True
3      True
      ...
17     True
18    False
19     True
20    False
Name: Data Structure Marks, Length: 21, dtype: bool
```

```
# True if the column has even one null value
print(df1['Data Structure Marks'].hasnans)

# StudentID is True too, because of the empty last row
print(df1['StudentID'].hasnans)
```

OUTPUT

```
True
True
```

**Note:** `hasnans` has **no brackets**. It works on a Series (one column), not on a whole DataFrame.

## 13.2 Removing missing values: dropna()

| Code | What it removes |
|---|---|
| `df.dropna()` | every row that has at least one missing value |
| `df.dropna(how='all')` | only rows where all values are missing |
| `df.dropna(subset=['col'])` | rows where the listed column(s) are missing |

```
df = pd.read_csv('student_data.csv')

# drops the rows that have any null value
df.dropna()
```

OUTPUT

```
   StudentID          FullName  ...  Instructor  Location
0     PH1001       Alif Rahman  ...   Mr. Karim     Dhaka
2     PH1003     Imran Hossain  ...   Mr. Karim     Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma    Sylhet
5     PH1006       Laila Begum  ...   Ms. Salma  Rajshahi
..       ...               ...  ...         ...       ...
14    PH1015      Wahiduzzaman  ...   Mr. Karim     Dhaka
16    PH1017       Afsana Mimi  ...   Mr. Karim     Dhaka
17    PH1018       Babul Ahmed  ...   Ms. Salma    Sylhet
19    PH1020        Nasir Khan  ...   Ms. Salma     Dhaka

[13 rows x 9 columns]
```

```
# delete a row only if all of its values are null (the empty last row)
df.dropna(how='all', inplace=True)

print(len(df))
```

OUTPUT

```
20
```

```
# delete the row if Python Marks is null
df.dropna(subset=['Python Marks'])
```

OUTPUT

```
   StudentID          FullName  ...  Instructor    Location
0     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
2     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
4     PH1005       Kamal Uddin  ...   Mr. Karim  Chattogram
..       ...               ...  ...         ...         ...
14    PH1015      Wahiduzzaman  ...   Mr. Karim       Dhaka
16    PH1017       Afsana Mimi  ...   Mr. Karim       Dhaka
17    PH1018       Babul Ahmed  ...   Ms. Salma      Sylhet
19    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka

[15 rows x 9 columns]
```

```
# same as before, but for 2 columns (a row is dropped if either one is null)
df.dropna(subset=['Python Marks', 'Algorithm Marks'])
```

OUTPUT

```
   StudentID          FullName  ...  Instructor  Location
0     PH1001       Alif Rahman  ...   Mr. Karim     Dhaka
2     PH1003     Imran Hossain  ...   Mr. Karim     Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma    Sylhet
5     PH1006       Laila Begum  ...   Ms. Salma  Rajshahi
..       ...               ...  ...         ...       ...
14    PH1015      Wahiduzzaman  ...   Mr. Karim     Dhaka
16    PH1017       Afsana Mimi  ...   Mr. Karim     Dhaka
17    PH1018       Babul Ahmed  ...   Ms. Salma    Sylhet
19    PH1020        Nasir Khan  ...   Ms. Salma     Dhaka

[13 rows x 9 columns]
```

**Note:** `dropna()` returns a new DataFrame; `df` itself changes only when you add `inplace=True`. That is why only the `how='all'` line above changed `df` (21 rows became 20).

## 13.3 Filling missing values: fillna()

```
df = pd.read_csv('student_data.csv')

# every null value is filled with 0
df.fillna(0)
```

OUTPUT

```
   StudentID          FullName  ...  Instructor    Location
0     PH1001       Alif Rahman  ...   Mr. Karim       Dhaka
1     PH1002     Fatima Akhter  ...   Ms. Salma  Chattogram
2     PH1003     Imran Hossain  ...   Mr. Karim       Dhaka
3     PH1004  Jannatul Ferdous  ...   Ms. Salma      Sylhet
..       ...               ...  ...         ...         ...
17    PH1018       Babul Ahmed  ...   Ms. Salma      Sylhet
18    PH1019      Faria Rahman  ...   Mr. David  Chattogram
19    PH1020        Nasir Khan  ...   Ms. Salma       Dhaka
20         0                 0  ...           0           0

[21 rows x 9 columns]
```

**Watch out:** `fillna(0)` fills **every** column, including text columns (the empty row gets `0` in `FullName`, which is not useful). It is better to fill each column with a sensible value.

```
# fill the nulls of the FullName column with 'unknown'
df['FullName'].fillna('unknown')
```

OUTPUT

```
0          Alif Rahman
1        Fatima Akhter
2        Imran Hossain
3     Jannatul Ferdous
            ...
17         Babul Ahmed
18        Faria Rahman
19          Nasir Khan
20             unknown
Name: FullName, Length: 21, dtype: str
```

```
# fill the nulls of Python Marks with the mean of Python Marks
df['Python Marks'] = df['Python Marks'].fillna(df['Python Marks'].mean())

df['Python Marks']
```

OUTPUT

```
0     88.000000
1     85.666667
2     85.000000
3     82.000000
        ...
17    85.000000
18    85.666667
19    89.000000
20    85.666667
Name: Python Marks, Length: 21, dtype: float64
```

**Correction:** the notes used `df['Python Marks'].fillna(df['Python Marks'].mean(), inplace=True)`. This is *chained assignment*. In current pandas it shows a `ChainedAssignmentError` warning and **does not change `df`**. Assign the result back instead: `df['col'] = df['col'].fillna(value)`.

**Note:** The mean ignores missing values. In this file the empty last row (row 20) is also filled with the mean. Remove it first with `dropna(how='all')` if you do not want that.

# 14. Duplicate Values

A row is a duplicate when all its values match an earlier row.

```
df = pd.DataFrame({
    "Name": ["Alice", "Bob", "Charlie", "Alice", "David", "Bob"],
    "City": ["New York", "London", "Paris", "New York", "Tokyo", "London"],
    "Score": [85, 90, 78, 70, 95, 90]
})

df
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     70
4    David     Tokyo     95
5      Bob    London     90
```

## 14.1 Finding duplicates: duplicated()

```
# duplicate rows show as True
print(df.duplicated())

# how many duplicates there are
print(df.duplicated().sum())
```

OUTPUT

```
0    False
1    False
2    False
3    False
4    False
5     True
dtype: bool
1
```

> **Note:** Row 3 (Alice, 70) is **not** a duplicate because its `Score` is different. Row 5 (Bob) matches row 1 exactly.

## 14.2 Removing duplicates: drop_duplicates()

```
# deleting duplicate rows (returns a new DataFrame)
df.drop_duplicates()
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     70
4    David     Tokyo     95
```

```
# permanent delete: changes df itself
df.drop_duplicates(inplace=True)

df
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     70
4    David     Tokyo     95
```

### 14.3 Choosing the columns to compare: subset

```
# duplicates decided by Name only
df.drop_duplicates(subset=['Name'])
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
4    David     Tokyo     95
```

```
# df itself is unchanged
df
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     70
4    David     Tokyo     95
```

```
# duplicates decided by Name and City together
df.drop_duplicates(subset=['Name', 'City'])
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
4    David     Tokyo     95
```

### 14.4 Which duplicate to keep: keep

| Value | Meaning |
|---|---|
| `keep='first'` | keep the first one, delete the later ones (default) |
| `keep='last'` | keep the last one, delete the earlier ones |

```
# keep the last value
df.drop_duplicates(subset=['Name'], keep='last')
```

OUTPUT

```
      Name      City  Score
1      Bob    London     90
2  Charlie     Paris     78
3    Alice  New York     70
4    David     Tokyo     95
```

```
df.drop_duplicates(subset=['Name'], keep='first')
```

OUTPUT

```
      Name      City  Score
0    Alice  New York     85
1      Bob    London     90
2  Charlie     Paris     78
4    David     Tokyo     95
```

## 15. Statistical Functions

These work on a column (Series). Missing values (`NaN`) are skipped automatically. (`describe()` is in Section 3.5. To work on all numeric columns at once, see Section 3.6.)

```
df = pd.read_csv('student_data.csv')
```

| Function | Meaning |
|---|---|
| `sum()` | total of the values |
| `min()` / `max()` | smallest / biggest value |
| `mean()` | average |
| `median()` | middle value after sorting |
| `mode()` | the value that appears most often |
| `std()` | standard deviation (how spread out the values are) |
| `corr()` | correlation between columns |

```
# sum
print(df['Data Structure Marks'].sum())
```

OUTPUT

```
1344.0
```

```
# max and min
print(df['Data Structure Marks'].min())
print(df['Data Structure Marks'].max())
```

OUTPUT

```
72.0
94.0
```

```
# mean -> average
print(df['Data Structure Marks'].mean())
```

OUTPUT

```
84.0
```

```
# median
print(df['Data Structure Marks'].median())
```

OUTPUT

```
85.5
```

```
# mode -> the number with the highest frequency
df['Data Structure Marks'].mode()
```

OUTPUT

```
0    75.0
1    85.0
2    86.0
3    88.0
Name: Data Structure Marks, dtype: float64
```

**Note:** `mode()` returns a Series because several values can appear equally often. Here 75, 85, 86 and 88 each appear twice.

```
# standard deviation -> std
print(df['Data Structure Marks'].std())
```

OUTPUT

```
6.501281924871945
```

```
# correlation matrix
df[['Data Structure Marks', 'Python Marks']].corr()
```

OUTPUT

```
                     Data Structure Marks  Python Marks
Data Structure Marks              1.000000      0.893455
Python Marks                      0.893455      1.000000
```

**Note:** Correlation is between -1 and +1. A value near +1 (here 0.89) means that when one mark goes up, the other usually goes up too. The diagonal is always 1 because each column matches itself perfectly.

### 15.1 Column-wise and row-wise sum

`axis=0` (default) works down each column. `axis=1` works across each row.

```
# sum of different columns, column wise
print(df[['Data Structure Marks', 'Python Marks']].sum())
```

OUTPUT

```
Data Structure Marks    1344.0
Python Marks            1285.0
dtype: float64
```

```
# sum of different columns, row wise
df[['Data Structure Marks', 'Python Marks']].sum(axis=1)
```

OUTPUT

```
0     173.0
1      92.0
2     173.0
3     160.0
      ...
17    173.0
18      0.0
19    175.0
20      0.0
Length: 21, dtype: float64
```

### 15.2 A total column with iloc and sum(axis=1)

```
# columns at positions 2, 3 and 4 (the three marks columns), summed row wise
df['Total Marks'] = df.iloc[:, 2:5].sum(axis=1)

df[['FullName', 'Total Marks']].head(6)
```

OUTPUT

```
           FullName  Total Marks
0       Alif Rahman        258.0
1     Fatima Akhter        184.0
2     Imran Hossain        261.0
3  Jannatul Ferdous        238.0
4       Kamal Uddin         95.0
5       Laila Begum        228.0
```

**Correction:** the notes used `df.iloc[::,2:5]`. The simple and correct form is `df.iloc[:, 2:5]` (all rows, columns 2 to 4).

**Difference from Section 11.2:** `+` gives `NaN` when any mark is missing, but `sum(axis=1)` skips missing values and counts them as 0. Fatima now has 184 (92 + 92) instead of `NaN`, and students with no marks at all (and the empty row) get a total of `0`.

## 16. apply()

`apply()` runs a function on every value of a Series, or on every row or column of a DataFrame. These examples continue with the `Total Marks` column from Section 15.2.

## 16.1 apply() with a lambda function

A `lambda` is a short one-line function. The example below does **min-max scaling**, which changes every value to a number between 0 and 1 using (x - min) / (max - min).

```
# min-max scaling. scaled values -> 0-1
mn = df['Total Marks'].min()
mx = df['Total Marks'].max()

df['Scaled Marks'] = df['Total Marks'].apply(lambda x: (x - mn) / (mx - mn))

df[['FullName', 'Total Marks', 'Scaled Marks']].head(6)
```

OUTPUT

```
           FullName  Total Marks  Scaled Marks
0       Alif Rahman        258.0      0.945055
1     Fatima Akhter        184.0      0.673993
2     Imran Hossain        261.0      0.956044
3  Jannatul Ferdous        238.0      0.871795
4       Kamal Uddin         95.0      0.347985
5       Laila Begum        228.0      0.835165
```

> **Note:** Here the smallest total is 0 (students with no marks) and the biggest is 273, so each scaled value is simply `Total Marks / 273`. The student with 273 gets 1.0 and the student with 0 gets 0.0.

## 16.2 apply() with your own function

```
# custom built function
def grading_system(marks):
    if marks >= 260:
        return 'A+'
    elif marks >= 250:
        return 'A'
    else:
        return 'A-'


df['Grade'] = df['Total Marks'].apply(grading_system)

df[['FullName', 'Total Marks', 'Grade']].head(6)
```

OUTPUT

```
           FullName  Total Marks Grade
0       Alif Rahman        258.0     A
1     Fatima Akhter        184.0    A-
2     Imran Hossain        261.0    A+
3  Jannatul Ferdous        238.0    A-
4       Kamal Uddin         95.0    A-
5       Laila Begum        228.0    A-
```

> **Common mistake:** pass the function name **without brackets**: `apply(grading_system)`. Writing `apply(grading_system())` calls the function immediately and raises a `TypeError`.

### 16.3 apply() on rows with axis=1

With `axis=1`, the function receives **one whole row** at a time, so it can use several columns together.

```
def marking_system(row):
    a = row['Data Structure Marks'] * 2
    b = row['Python Marks'] * 3
    c = row['Algorithm Marks'] * 4

    return a + b + c


# the function runs once for every row
df['Exceptional Marks'] = df.apply(marking_system, axis=1)

df[['FullName', 'Exceptional Marks']].head(6)
```

OUTPUT

```
           FullName  Exceptional Marks
0       Alif Rahman              774.0
1     Fatima Akhter                NaN
2     Imran Hossain              783.0
3  Jannatul Ferdous              714.0
4       Kamal Uddin                NaN
5       Laila Begum              684.0
```

**Correction:** the notes named the function's input `df`. Inside the function it is only one row, not the whole DataFrame, so `row` is a clearer name. If any of the three marks is missing, the result is `NaN`.

## 17. Dates and Time Differences

Dates read from a CSV file are plain text, so you cannot do date calculations on them. Convert them with `pd.to_datetime()` first.

```
df = pd.read_csv('student_completed_data.csv')

# dtype is text (object in older pandas) before converting
print(df['EnrollmentDate'].dtype)
```

OUTPUT

```
str
```

### 17.1 Converting text to dates

```
# converted to the datetime64 type
df['EnrollmentDate'] = pd.to_datetime(df['EnrollmentDate'])

print(df['EnrollmentDate'].dtype)
```

OUTPUT

```
datetime64[us]
```

**Note:** The type prints as `datetime64[us]` in pandas 3 and as `datetime64[ns]` in older versions.

### 17.2 Getting the year and day

After conversion, the `.dt` accessor gives parts of the date, for example `.dt.year` and `.dt.day`.

```
# take the year out and create a column
df['Enrollment Year'] = df['EnrollmentDate'].dt.year

# take the day out and create a column
df['Enrollment Day'] = df['EnrollmentDate'].dt.day

df[['FullName', 'EnrollmentDate', 'Enrollment Year', 'Enrollment Day']].head(4)
```

OUTPUT

```
           FullName EnrollmentDate  Enrollment Year  Enrollment Day
0       Alif Rahman     2024-09-01             2024               1
1     Fatima Akhter     2024-08-11             2024              11
2     Imran Hossain     2024-05-08             2024               8
3  Jannatul Ferdous     2024-07-05             2024               5
```

> **Correction:** the notes named the day column `Enrollment Date`, which is easy to mix up with the `EnrollmentDate` column. It is renamed `Enrollment Day` here.

### 17.3 Time difference (timedelta)

Subtracting two datetime columns gives a **time difference** in days.

```
df['FinishedDate'] = pd.to_datetime(df['FinishedDate'])

df['Total time taken to finish'] = df['FinishedDate'] - df['EnrollmentDate']

df[['FullName', 'EnrollmentDate', 'FinishedDate', 'Total time taken to finish']].head(4)
```

OUTPUT

```
           FullName EnrollmentDate FinishedDate Total time taken to finish
0       Alif Rahman     2024-09-01   2024-12-23                  113 days
1     Fatima Akhter     2024-08-11   2024-11-07                   88 days
2     Imran Hossain     2024-05-08   2024-08-08                   92 days
3  Jannatul Ferdous     2024-07-05   2024-09-23                   80 days
```

> **Note:** Both columns must be converted to datetime first. If one of them is still text, the subtraction raises a `TypeError`.

## 18. groupby()

`groupby('column')` splits the rows into groups that share the same value in that column. You then calculate something for each group. These examples continue with the `df` from Section 17.

### 18.1 Setting up some missing values

The class set three `Total Marks` values to `NaN` so that they can be filled later.

```
import numpy as np

# some values are set to null
df.loc[df['FullName'] == 'Afsana Mimi', 'Total Marks'] = np.nan
df.loc[df['FullName'] == 'Sadia Chowdhury', 'Total Marks'] = np.nan
df.loc[df['FullName'] == 'Alif Rahman', 'Total Marks'] = np.nan

print(df['Total Marks'].isnull().sum())
```

OUTPUT

```
3
```

## 18.2 Creating the groups

```
# groups are created based on the Instructor column
instructor_group = df.groupby('Instructor')

print(type(instructor_group))
```

OUTPUT

```
<class 'pandas.api.typing.DataFrameGroupBy'>
```

> **Note:** The result is a `DataFrameGroupBy` object, not a table. It only holds the groups. You see results after you add a calculation like `min()` or `mean()`.

## 18.3 Looping through the groups

Each step of the loop gives a **key** (the group name) and a **DataFrame** (the rows of that group).

```
for instructor, df_ins in instructor_group:
    print(instructor)
    print(df_ins)
```

OUTPUT

```
Mr. David
   StudentID        FullName  ... Enrollment Day Total time taken to finish
4     PH0005     Kamal Uddin  ...              1                    76 days
5     PH0006     Laila Begum  ...              7                   115 days
6     PH0007  Mahmudul Hasan  ...              7                    43 days
9     PH0010    Priya Sharma  ...             19                    61 days
12    PH0013    Tanvir Ahmed  ...              7                    57 days
13    PH0014      Urmi Akter  ...              4                    79 days
14    PH0015    Wahiduzzaman  ...             22                    84 days
16    PH0017     Afsana Mimi  ...             18                    58 days
17    PH0018     Babul Ahmed  ...             23                    89 days

...
```

*(Output shortened: there is one block for each instructor.)*

> **Correction:** the notes used `display(df_ins)`. `display` works only inside Jupyter, so `print(df_ins)` is used here. In Jupyter, `display(df_ins)` prints a nicer table.

## 18.4 Calculations for each group

```
# minimum of every column in the whole DataFrame
print(df.min())
```

OUTPUT

```
StudentID                              PH0001
FullName                         Afsana Mimi
CompletionStatus                   Completed
EnrollmentDate           2024-01-04 00:00:00
                                       ...
Total Marks                              234.0
Enrollment Year                           2024
Enrollment Day                               1
Total time taken to finish       40 days 00:00:00
Length: 11, dtype: object
```

```
# the minimum of every column for each group
instructor_group.min()
```

OUTPUT

```
           StudentID      FullName  ... Enrollment Day Total time taken to finish
Instructor                          ...
Mr. David     PH0005   Afsana Mimi  ...              1                    43 days
Mr. Karim     PH0002  Faria Rahman  ...              4                    80 days
Ms. Salma     PH0001   Alif Rahman  ...              1                    40 days

[3 rows x 10 columns]
```

**Watch out:** `min()` on text columns (`FullName`, `Location`) compares letters in alphabetical order, so those values are not meaningful. Select the column you need: `instructor_group['Total Marks'].min()`.

```
# count, mean, std, min, quartiles and max for each group
instructor_group.describe()
```

OUTPUT

```
           EnrollmentDate                                ... Total time taken to finish  \
                    count                           mean  ...                        max
Instructor                                               ...
Mr. David               9           2024-04-22 13:20:00  ...          115 days 00:00:00
Mr. Karim               7  2024-05-30 03:25:42.857142  ...          120 days 00:00:00
Ms. Salma               4           2024-05-13 18:00:00  ...          113 days 00:00:00


                                std
Instructor
Mr. David   21 days 12:42:45.670041
Mr. Karim   17 days 01:52:40.910298
...
```

*(Output shortened: the real table has 48 columns, one block of statistics for each numeric or date column.)*

```
# keep the mean Total Marks of each instructor
mean_marks = instructor_group['Total Marks'].mean()

mean_marks
```

OUTPUT

```
Instructor
Mr. David    270.375000
Mr. Karim    274.166667
Ms. Salma    249.666667
Name: Total Marks, dtype: float64
```

## 18.5 Filling missing values with the group average

`df['Instructor'].map(mean_marks)` replaces each instructor's name with that instructor's average mark. `fillna()` then uses these averages only where `Total Marks` is missing.

```
# for a null value, use the mean of that row's Instructor (matched with map)
df['Total Marks'] = df['Total Marks'].fillna(df['Instructor'].map(mean_marks))

df.loc[[0, 11, 16], ['FullName', 'Instructor', 'Total Marks']]
```

OUTPUT

```
           FullName Instructor  Total Marks
0       Alif Rahman  Ms. Salma   249.666667
11  Sadia Chowdhury  Mr. Karim   274.166667
16      Afsana Mimi  Mr. David   270.375000
```

**Note:** The mean skips missing values. For example Ms. Salma's mean (249.67) is the average of her 3 students who have marks (269, 246 and 234), because Alif Rahman's mark is missing.